SOFTWARE ARCHITECTURE DESIGN · 2025/26

Music Playlist Manager

Presentazione finale

Gruppo 02

Apicella Francesco
Campagna Lorenzo
Cutolo Raffaele
Gioia Errico

18 giugno 2026

Discussione finale

Un Music Playlist Manager desktop

Applicazione desktop in Java / JavaFX per creare e gestire playlist con riproduzione simulata e filtri avanzati. Sviluppata in tre sprint con processo Scrum, a partire dalle User Epic fornite.

24
User Stories

54
Story Points

3
Sprint

Copertura del backlog

Alta	Operazioni base e riproduzione (US-01 … US-11)
Media	Undo, home, generazione, tag (US-12 … US-16)
Bassa	Riordino e persistenza (US-17 … US-18)

Il design complessivo del sistema

Architettura a livelli con alta coesione e basse dipendenze: la view dipende dai controller, i controller dai service, mai dal Model.

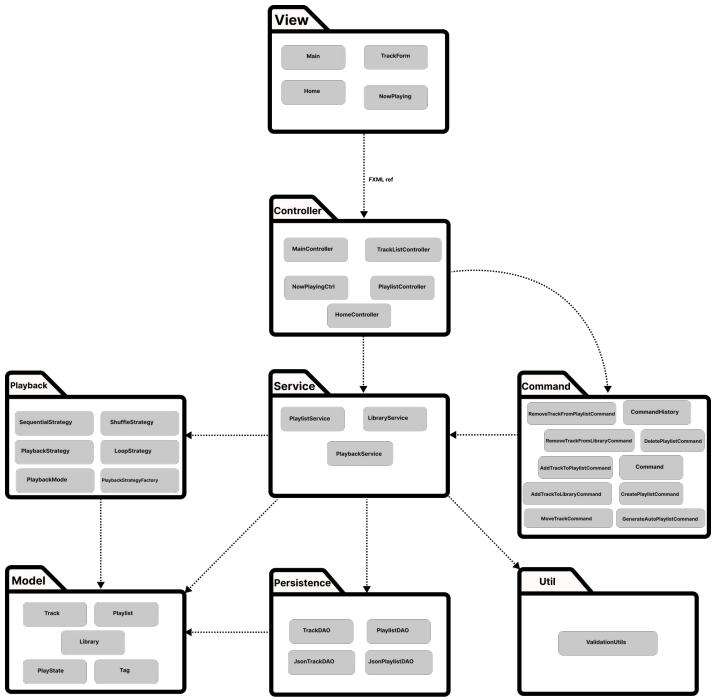


Diagramma dei package

Good design principles

Coesione	un package per responsabilità
Accoppiamento	controller → service, mai il Model
Astrazione	interfacce Strategy e DAO
DRY	factory unica + validazione centrale
Pattern	solo dove risolvono un problema

Cinque design pattern, dove servono

Ogni pattern è stato introdotto solo dove risolve un problema concreto del dominio.

Strategy

Le tre modalità di riproduzione (sequenziale, shuffle, loop) intercambiabili a runtime.

Command

Ogni operazione mutante è incapsulata e annullabile tramite la CommandHistory.

Observer

UI aggiornata in tempo reale tramite le property osservabili di JavaFX.

Factory

Creazione delle strategie di riproduzione centralizzata in un unico punto di estensione.

DAO

Persistenza su disco isolata (JSON con Jackson) e facilmente sostituibile.

Cosa mostriamo nella demo

Tutte le 24 user stories sono “Done”. Di seguito i blocchi della demo dal vivo.

Libreria & playlist

Creazione, modifica, eliminazione di tracce; organizzazione in playlist.

Riproduzione simulata

Play, pausa e skip su tracce e playlist; tre modalità di ascolto.

UI in tempo reale

Pannello Now Playing e traccia corrente evidenziata, aggiornati live.

Undo

Annullamento di aggiunta, rimozione e riordino tramite CommandHistory.

Home & generazione auto

Tracce e playlist più riprodotte; playlist automatiche per genere, anno o tag.

Persistenza

Salvataggio alla chiusura e caricamento automatico all'avvio (JSON).

Sprint 3 — chiusura dello scope

Sprint finale: 18 SP pianificati, tutte le user stories completate. Atterraggio del burndown a 0.

18
SP completati

8
User stories

8/8
US “Done”

0
SP residui

User stories dello Sprint 3

US-13	Home · più riprodotti
US-14a, US-14b	Auto-playlist · genere/anno
US-15	Tag visivi sulle tracce
US-16	Auto-playlist · per tag
US-17	Riordino tracce (annullabile)
US-18a, US-18b	Persistenza JSON (save/load)

Tre sprint, ritmo costante

Pre-game per backlog iniziale, architettura e setup; poi tre sprint con planning, daily, review e retrospective.

3
Sprint completati

18
Velocity media (SP)

4
Membri del team

8/9h
a settimana / membro

Cadenza dello sprint

Planning	1.5h · scelta US e task
Daily	5' · fatto / da fare / ostacoli
Sviluppo	integrazione frequente sul branch
Review	code review incrociata 30'
Chiusura	Review 20' + Retrospective 20'

54 Story Points su 54

Velocity 12 → 24 → 18: lo scostamento prudente del primo sprint è stato riassorbito già nel secondo. L'intero scope pianificato è chiuso.

54
SP pianificati

54
SP completati

18
Velocity media

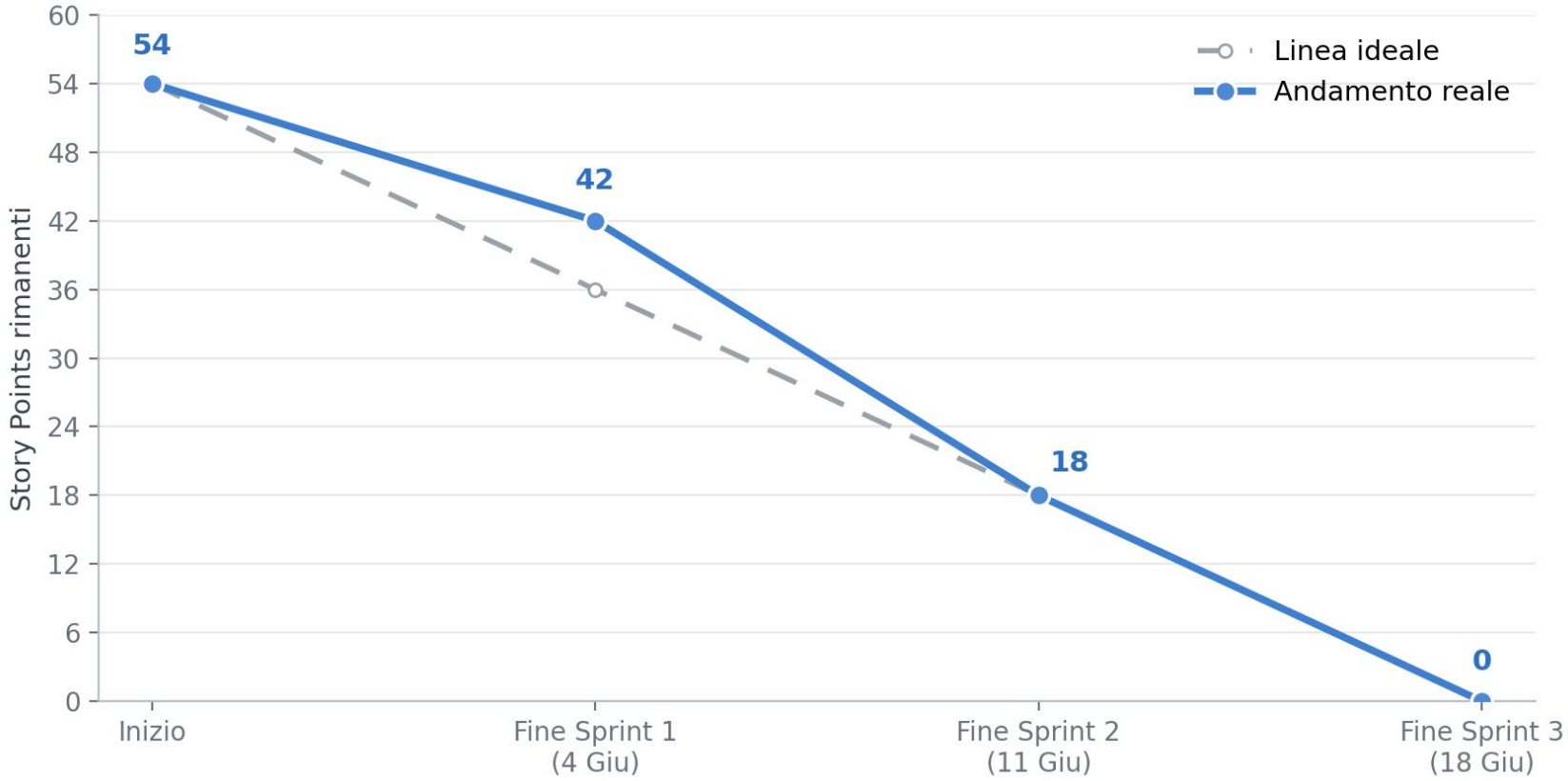
24/24
User Stories “Done”

Velocity per sprint

Sprint 1	12 SP · stima prudente
Sprint 2	24 SP · scostamento riassorbito
Sprint 3	18 SP · scope completato
Media	18 SP / sprint

Story Points residui sull'intero backlog

A fine progetto i punti residui scendono a 0: l'andamento reale converge sulla linea ideale.



L'arco delle retrospettive

Ogni retrospettiva ha alimentato la successiva: il disallineamento documento↔codice, emerso nello Sprint 1, è stato chiuso nello Sprint 3.

Sprint 1 — Emerso
Disallineamento documento↔codice (PlaylistController).

Sprint 2 — Confermato
Ricorre su util/ValidationUtils; carichi oltre la capacity.

Sprint 3 — Chiuso
Diagramma allineato al codice contestualmente: impegno mantenuto.

Keep doing	Design collettivo, code review a rotazione, integrazione frequente, diagramma allineato.
More of	Pair programming sulle componenti di infrastruttura condivisa (serializzazione del Model).
Start	Tenere un registro delle decisioni di design e dei raffinamenti di backlog.
Less of	Pianificare carichi oltre la capacity nominale: gestiti a buffer, ma da contenere.
Stop	Aggiornare il documento di architettura solo a posteriori: chiuso in questo sprint.

Cosa ci ha insegnato Scrum

Tre lezioni sul processo che ci portiamo dietro, al di là del prodotto.

Stimare è una competenza

La stima migliora solo misurando gli sprint passati: 12 → 24 → 18 SP, fino a una velocity realistica su cui pianificare.

Il time-boxing è design

Decidere cosa non fare, e saperlo motivare, vale quanto scrivere codice: l'undo sulla modifica è stato escluso consapevolmente.

La documentazione vive col codice

Un documento di architettura ha senso solo se evolve con l'implementazione: tenerlo “per dopo” è documentazione sbagliata.

Scope chiuso, idee per il futuro

Progetto completato nei tempi rispettando il time-boxing. Alcune estensioni valutate restano come lavoro futuro.

Undo sulla modifica

Estendere l'annullamento anche alla modifica di una traccia: valutata in corso di sprint e rinviata per time-boxing.

Redo

Affiancare all'undo la possibilità di ripetere un'operazione annullata, sfruttando la CommandHistory esistente.

Riproduzione audio reale

Sostituire la riproduzione simulata con un playback audio effettivo, mantenendo invariate le strategie esistenti.

Grazie per l'attenzione.

github.com/fra343/Gruppo2-SAD
trello.com/b/Mpb9bjYE/sad